Software Project Management – IS212

Software Development Processes

Software Development Processes

Objectives

On completing this module, you will be able to:

- Summarise the key traditional and agile software development processes
- Identify the pros and cons of each

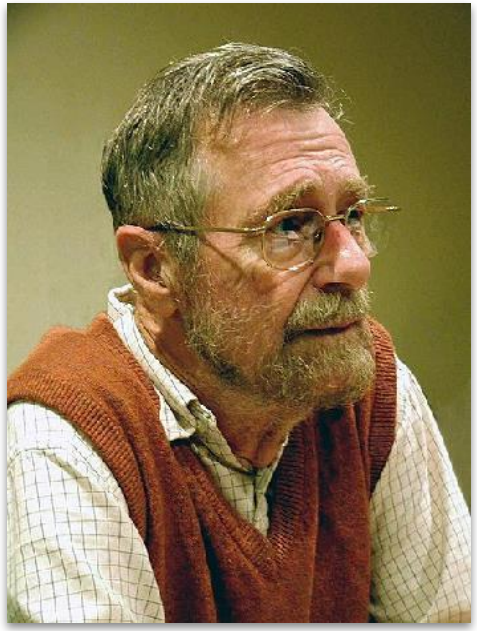
Traditional

- Waterfall
- RUP

Agile

- Extreme Programming
- Kanban
- Scrum

1960/70s – Software Crisis



The major cause of the software crisis is that the machines have become several orders of magnitude more powerful!

To put it quite bluntly: as long as there were no machines, programming was no problem at all; when we had a few weak computers, programming became a mild problem, and now we have gigantic computers, programming has become an equally gigantic problem.

– Edsger Dijkstra

projects running over time

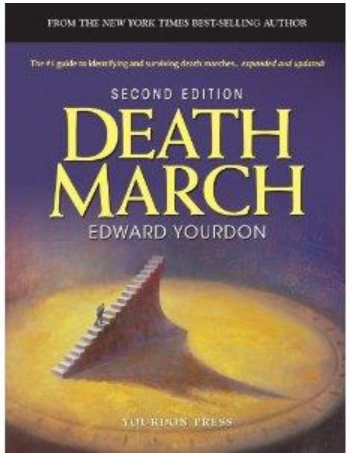
projects running over budget

software often didn't meet requirements

Computing power was outpacing the ability of software developers to effectively use it

code that was difficult to maintain

software never being delivered





Software requirements may not be 'frozen' before development

Requirements are constantly changing

Software can be subject to (frequent) change

How does engineering **software** compare to, say, engineering a **bridge**?

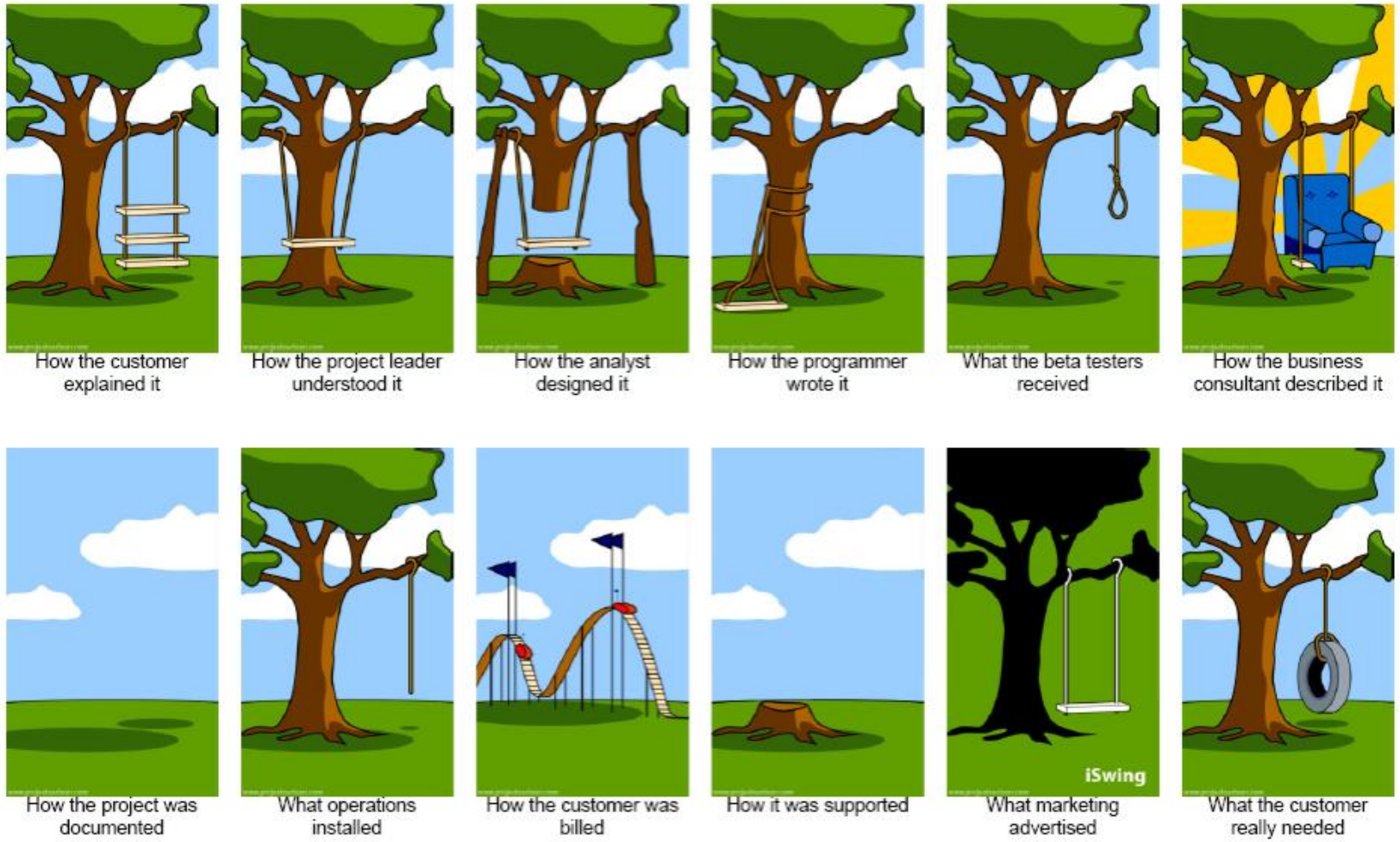
*Software and Work In Progress (WIP) are **"invisible"***

Most of the time, software development is not visible until the end product is out.

Technical debt is a major challenge

Debt builds up due to time pressure, revisiting the c
becomes a major challenge

Stakeholders can be on different wavelengths…



Software Development Processes

- Structured software development processes have been established to improve quality, efficiency, and predictability in software projects

- We will study five key processes (focusing on scrum in subsequent weeks)

Traditional

- Waterfall
- RUP

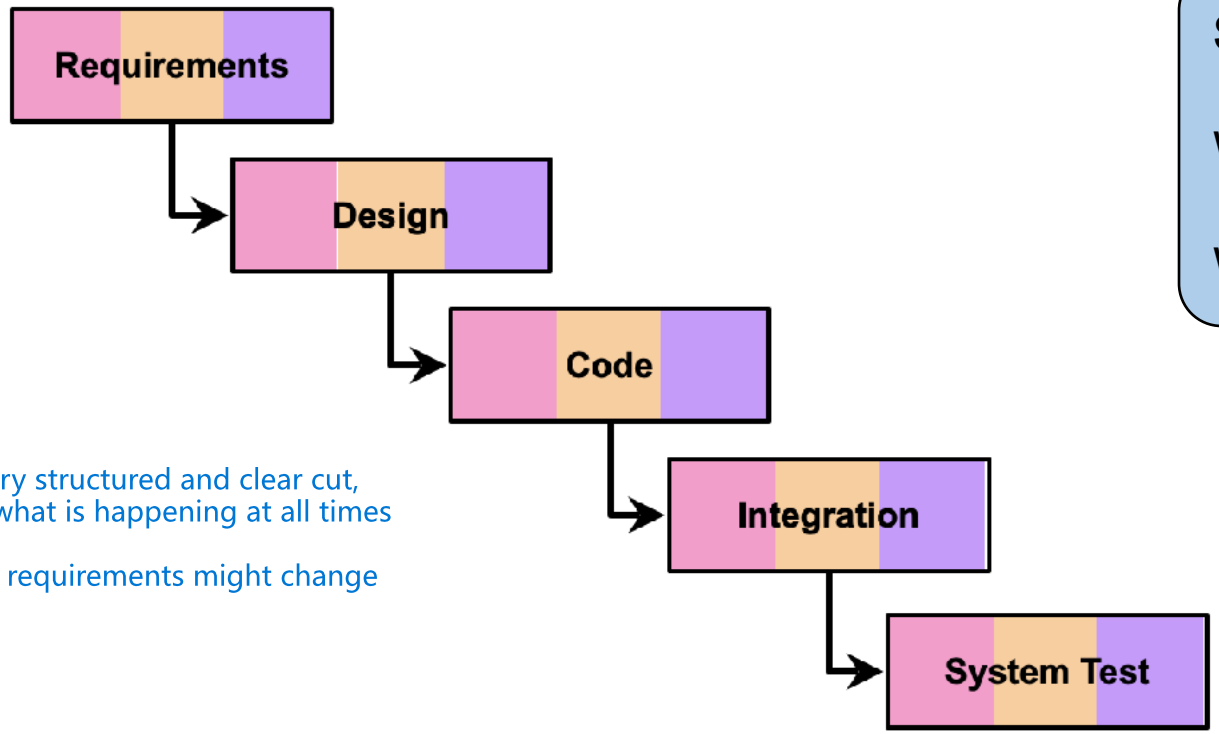
Agile

- Extreme Programming
- Kanban
- Scrum



Waterfall

Idea: break software development into linear, sequential phases



Simple, intuitive, and appealing?

When might it work well? ✅

When might it not? ❌

Advantages: very structured and clear cut, knows exactly what is happening at all times

Disadvantages: requirements might change after designing

Waterfall – Discussion

- Don't knock it! Waterfall is extensively used and often works well –
  - If the project is of small to medium-scope (relatively short duration required)
  - If requirements are contractually agreed before development (e.g. when outsourcing to vendors)
  - If the product definition is stable and unlikely to evolve over time
  - If the industry is regulated (strict compliance with standards in healthcare, aerospace, defence, …)
- However – waterfall's inflexible partitioning of phases makes it less appropriate for projects that involve a lot of uncertainty
- Harder to respond to changing customer requirements
- Difficult to experiment, which may stifle the team's innovation
- Testing only at the end of the process invites failure. **(Why?)**

Alternative – Develop Software Iteratively

- Iterative processes attempt to mitigate the rigidity of the waterfall process

- More flexibility

- Enhanced stakeholder collaboration

- Early detection of issues

- Development is broken down into smaller, manageable increments



Image source: https://www.pacific-research.com/iterative-product-development/

Software Development Processes

Traditional

Agile

iterative traditional method —>

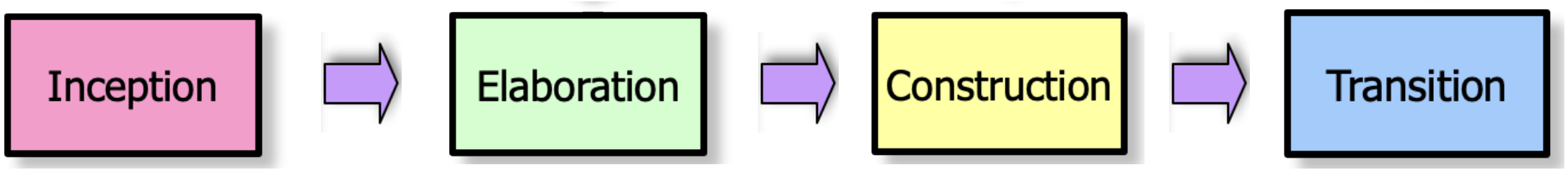
- Waterfall
- RUP

iterative agile methods —>

- Extreme Programming
- Kanban
- Scrum

Rational Unified Process (RUP)

- The Rational Unified Process (RUP) consists of four phases
  - i.e. key 'milestones' for the project, similar to waterfall



understand ***what*** product to build

understand ***how*** to build the product

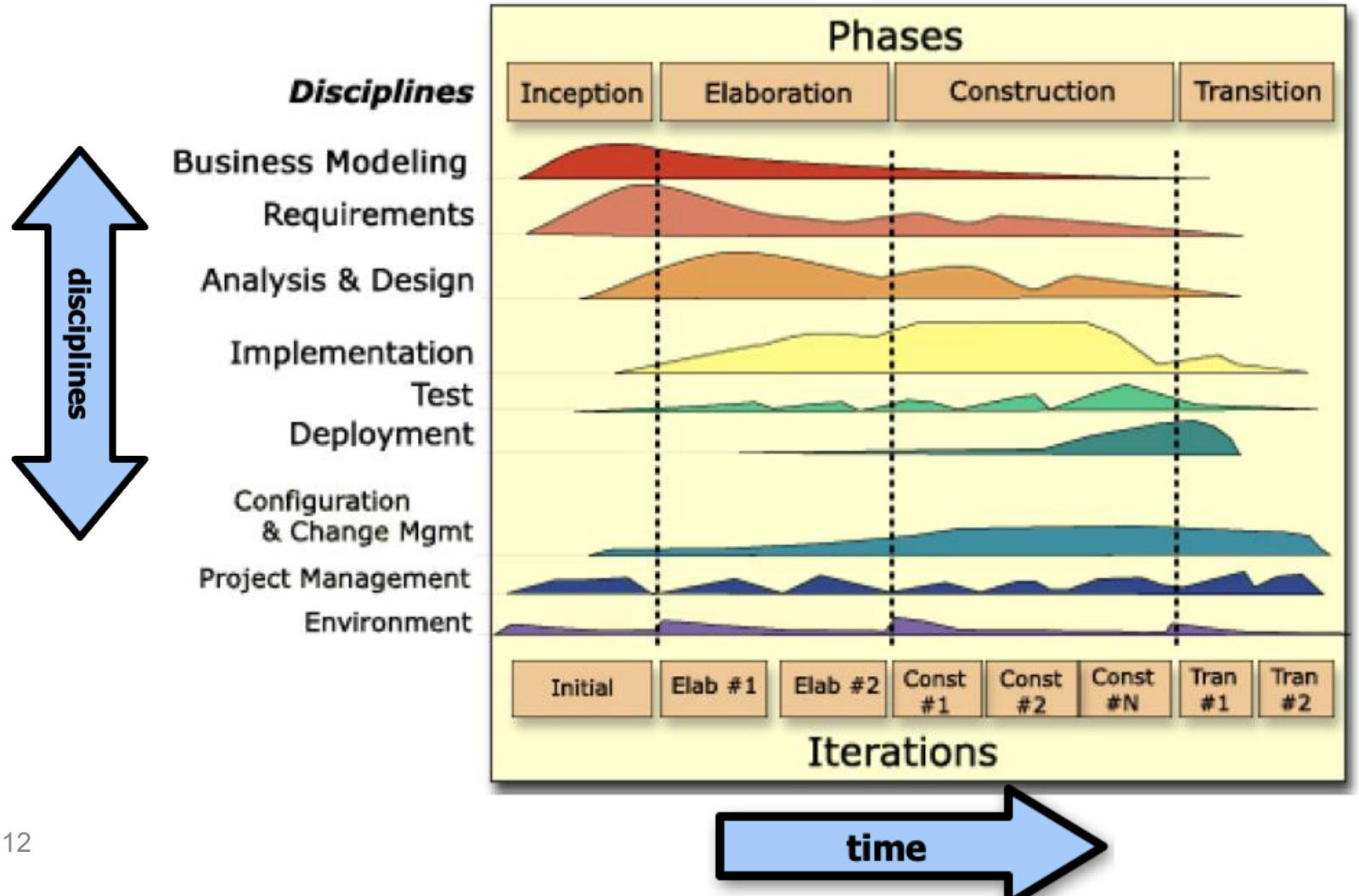
actually ***build*** the product

validate and ***deploy*** the product

- Various 'engineering disciplines' then cut across all four phases
  - *e.g. business modelling, requirements, implementation, testing*

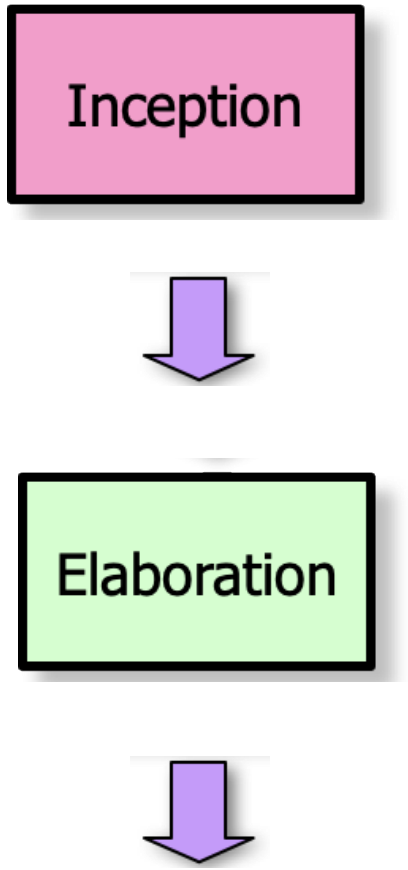
Rational Unified Process (RUP)

- Unlike waterfall – it explicitly encourages iterations within phases



Challenge: almost everyone is involved in all phases, compared to waterfall approach

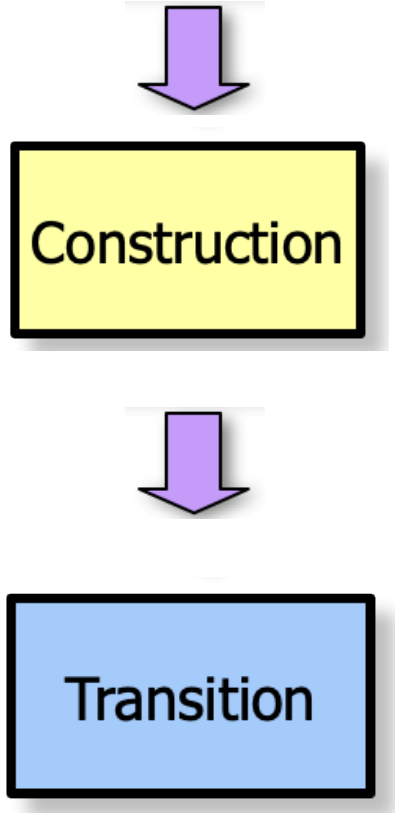
RUP phases – cross-discipline outcomes



- Vision document / business case
- Develop high-level requirements / use cases
- Stakeholder concurrence; identify risks

- “80% complete” use case models
- Prototypes for exploring key risks
- Development plan; architecture description

RUP phases – cross-discipline outcomes



- Demonstrable prototypes
- Working software components
- Bulk of coding

- Training end users / maintainers
- Evaluate against ‘Inception’ requirements
- ‘Post-mortem’ project analysis

RUP – Discussion

- Support for iteration a big advantage over classic waterfall
- Forces integration to happen throughout the software development
- Testing can happen as early as 'inception' – not left until the end!
- But – RUP is quite complex; a lot of process 'overhead'
  - Less flexible than agile processes *(see later!)*

SMU Classification: Restricted

Software Development Processes

Traditional → **Agile**

- Waterfall
- RUP

- Extreme Programming
- Kanban
- Scrum

Agile – Motivation

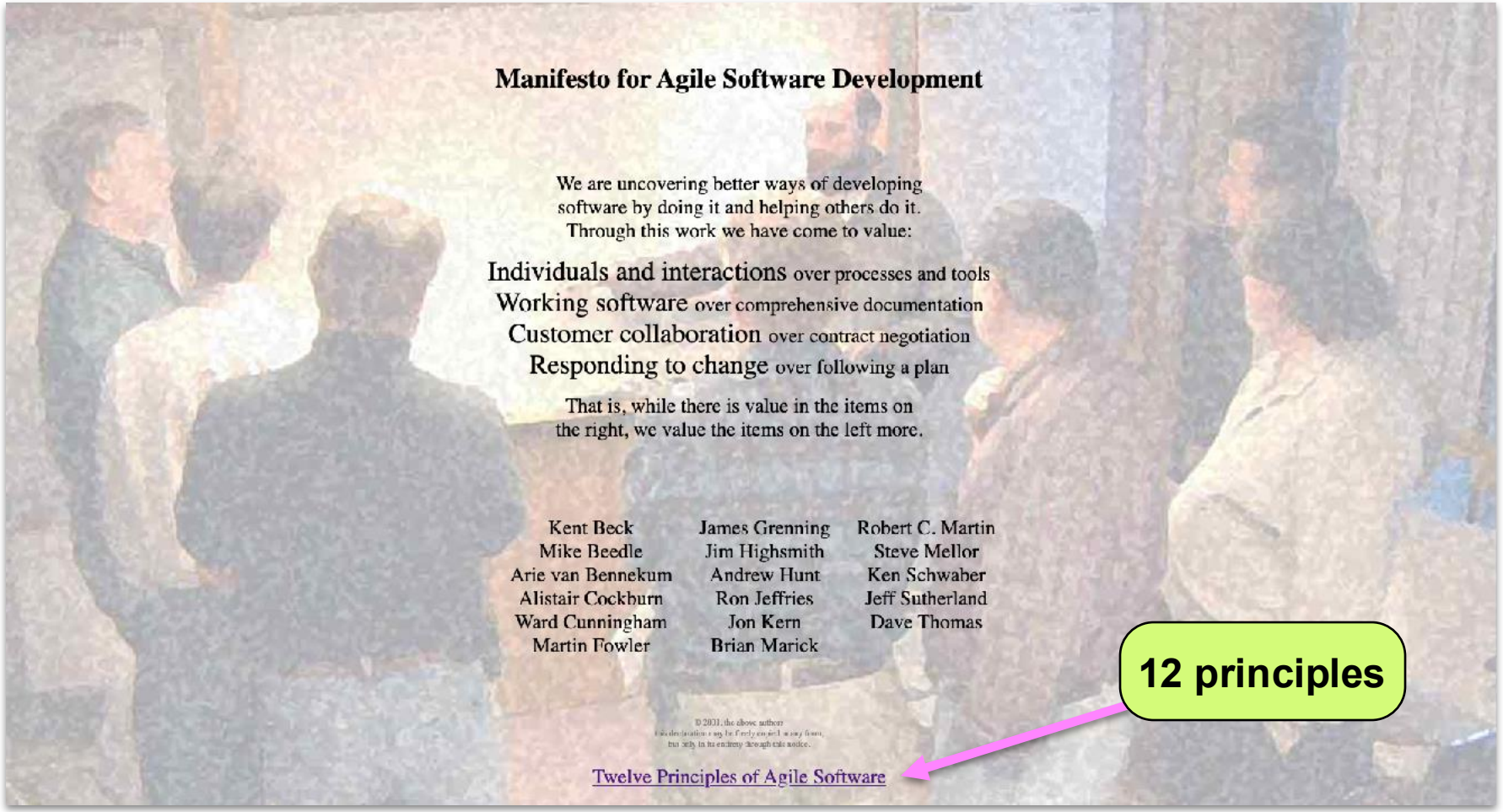
- Traditional software processes can be overly regulated, planned, and micromanaged, i.e. bureaucratic

- In reaction, lightweight software processes started evolving that collectively became known as "agile", emphasising:

Individuals and interactions	**over**	processes and tools
Working software		comprehensive documentation
Customer collaboration		contract negotiation
Responding to change		following a plan

"while there is value in the items on the right, we value the items on the left more"

Agile Manifesto (2001)



https://agilemanifesto.org/

Agile Principles *(re-organised; Meyer '14)*

Agile principles

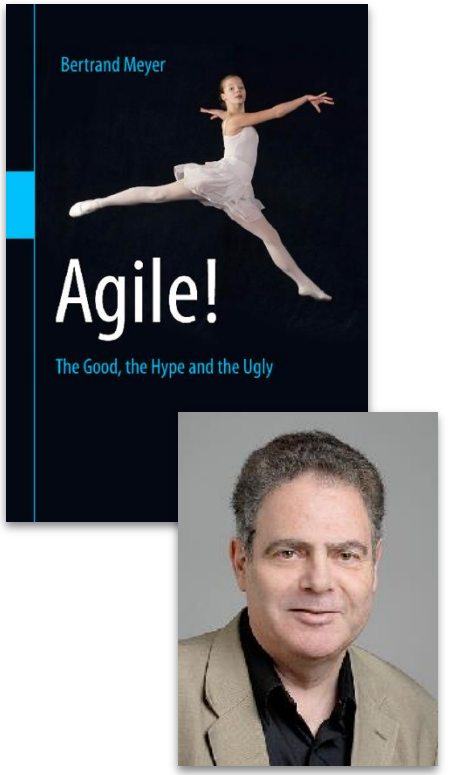
Organizational

1 Put the customer at the center.
2 Let the team self-organize.
3 Work at a sustainable pace.
4 Develop minimal software:
  4.1 Produce minimal functionality.
  4.2 Produce only the product requested.
  4.3 Develop only code and tests.
5 Accept change.

Technical

6 Develop iteratively:
  6.1 Produce frequent working iterations.
  6.2 Freeze requirements during iterations.
7 Treat tests as a key resource:
  7.1 Do not start any new development until all tests pass.
  7.2 Test first.
8 Express requirements through scenarios.

key insight



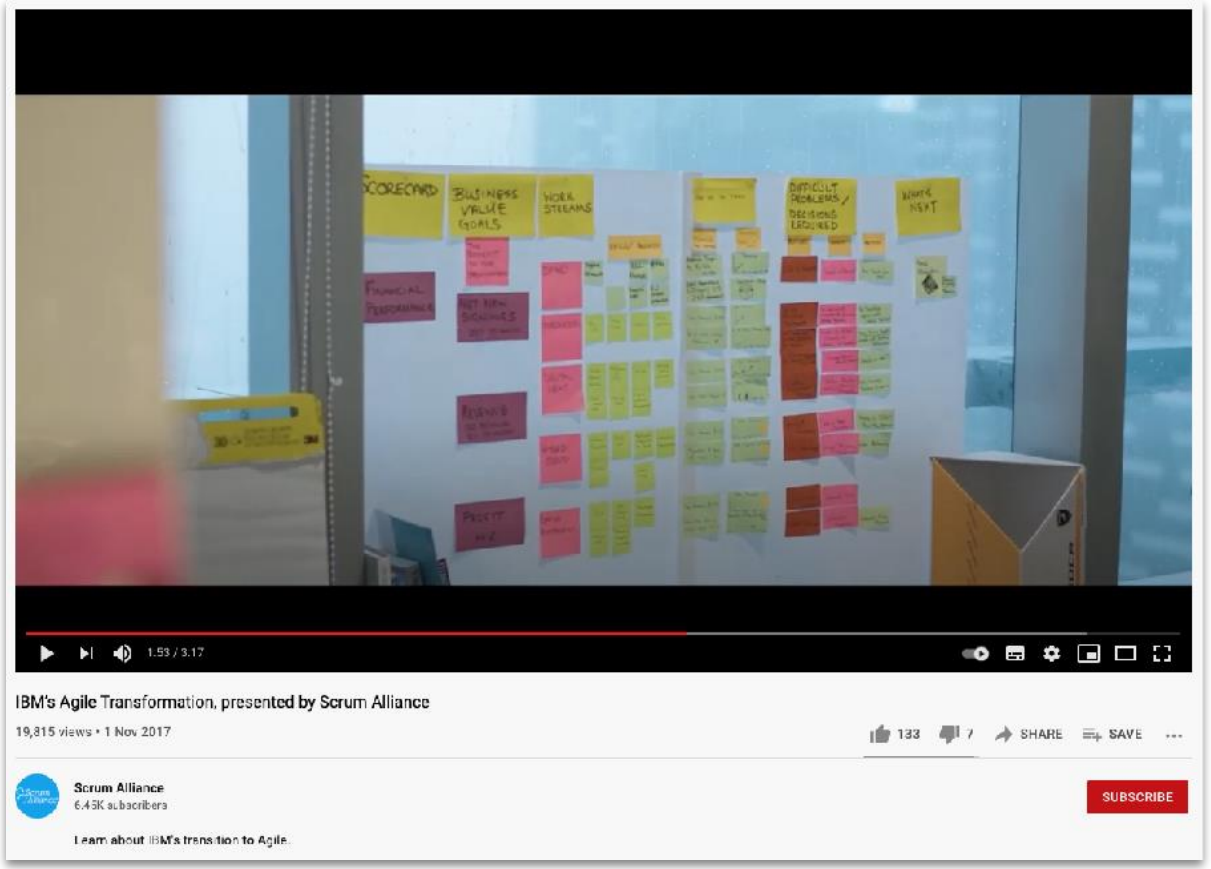
Bertrand Meyer

Agile – IBM Video

Padlet Exercise

What agile tools and methods do you see IBM using?

https://padlet.com/XXX/YYY



https://www.youtube.com/watch?v=Xu0nxyebc6g

Highly iterative and flexible

Working software delivered frequently

Working software (not "WIP") is the measure of progress

Agile – Key Characteristics

Late changes in requirements – no problem

Close, daily co-operation

Visualise "WIP" to manage team's capacity and highlight constraints

Software Development Processes

Traditional

- Waterfall
- RUP

Agile

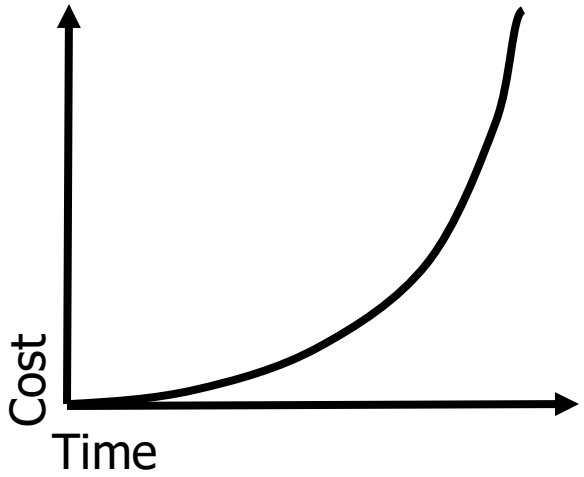
- Extreme Programming
- Kanban
- Scrum



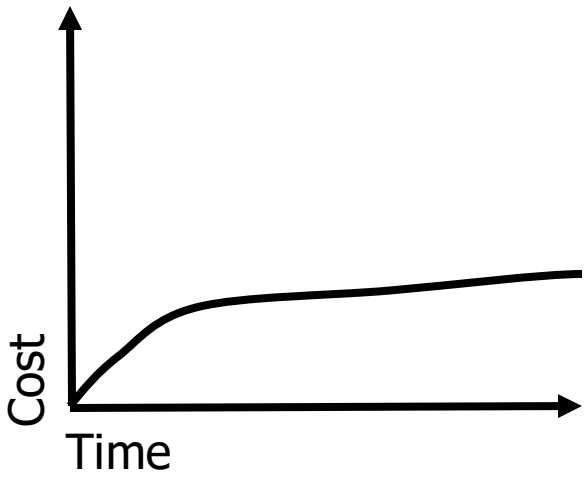
Extreme Programming (XP)

- Premise of extreme programming (XP)

Traditional
- upfront design
- code late
- release when ‘done’



XP
- continuous design
- code early
- release early



Extreme Programming (XP)

- eXtreme programming (XP) advocates frequent releases in short development cycles
- **Idea –** take good software development practices to a logical extreme
- Code reviews are beneficial…

  *…so review code continuously **(pair programming)***
- Automated testing is beneficial…

  *…so write your tests before you write your code **(Test-Driven Development)***

  Write test cases first based on the user's requirements

Extreme Programming (XP)

- XP is based on four principles

Coding is the core activity	Code doesn't only deliver the solution – use it to explore / explain problems too!
LOTS of testing during dev	Design unit tests or software contracts first – *then* code! (aka **Test-Driven Development**)
Developers and customers communicate directly	Programmer must understand the business requirements to design a technical solution
Regular refactoring	Overall system design is considered during regular **refactoring** exercises

SMU Classification: Restricted

12 XP Practices

Fine-scale feedback

- Pair programming
- Planning game
- Test-Driven Development
- Whole team

Shared understanding

- Coding standard
- Collective code ownership
- Simple design
- System metaphor

Continuous process

- Continuous integration
- Design refactoring
- Small releases

Programmer welfare

- Sustainable pace

XP – Impact

- XP provided a ‘jolt’ that brought attention to agile methods
- Some see XP as ‘dogmatic’; others see it as a consistent, strong view of how programming should be practiced
- Many of the individual practices promoted by XP have left their mark on industry, especially that:
  - Projects should integrate code all the time
  - Tests are a key resource, and should be run against code often
- XP practices like test-driven development can be integrated with other software development processes (e.g. scrum)

Software Development Processes

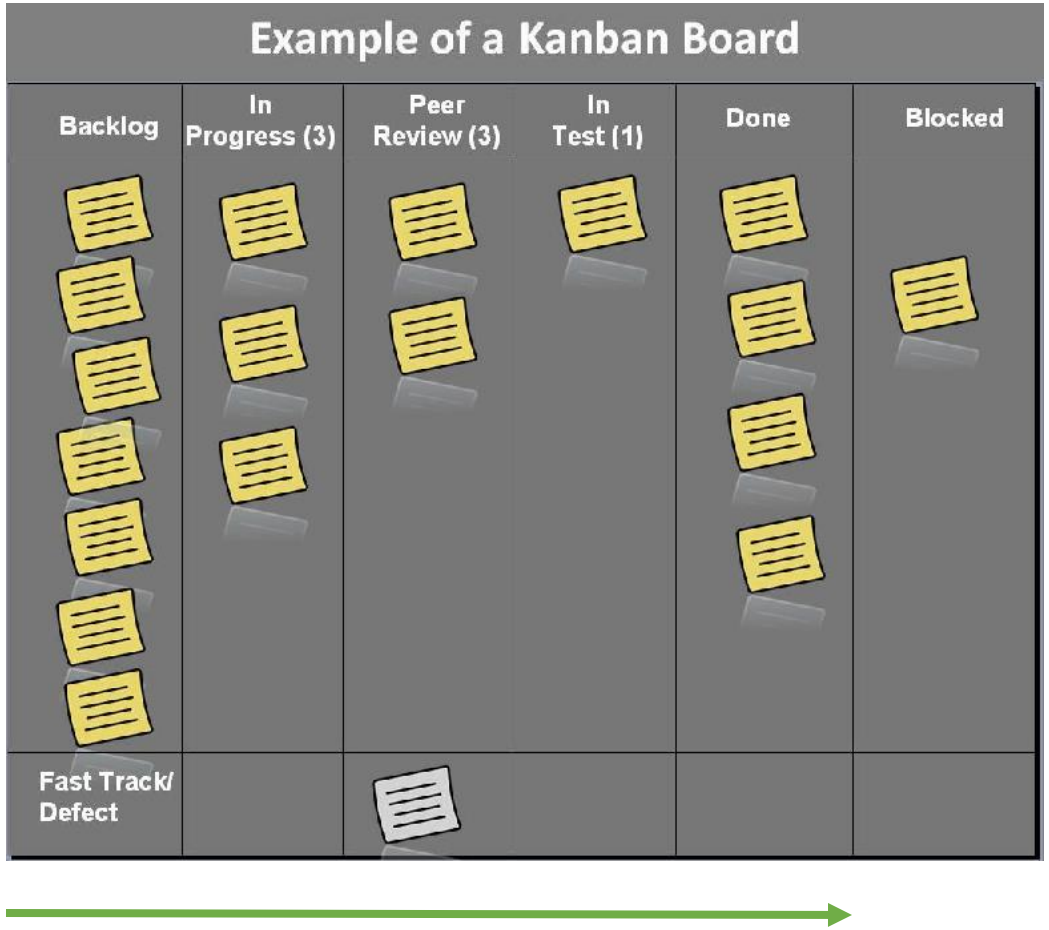
Traditional

- Waterfall
- RUP

Agile

- Extreme Programming
- Kanban
- Scrum

Kanban



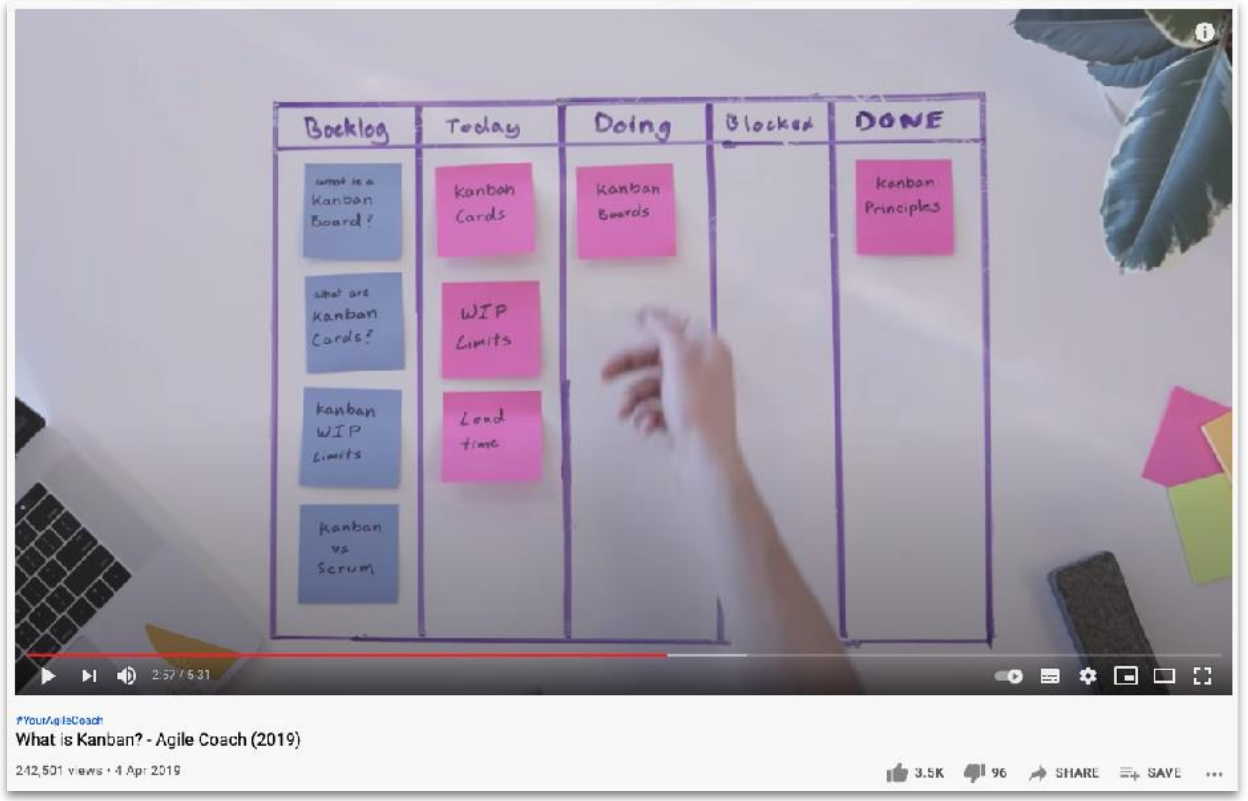
*"**Kanban**"*

- Japanese word meaning (roughly)"large visual board"

- *idea: visualising a team's WIP helps to track workflow and identify constraints*

- *NB: in the Kanban process, a single board is used **continuously** throughout the whole project*

Kanban

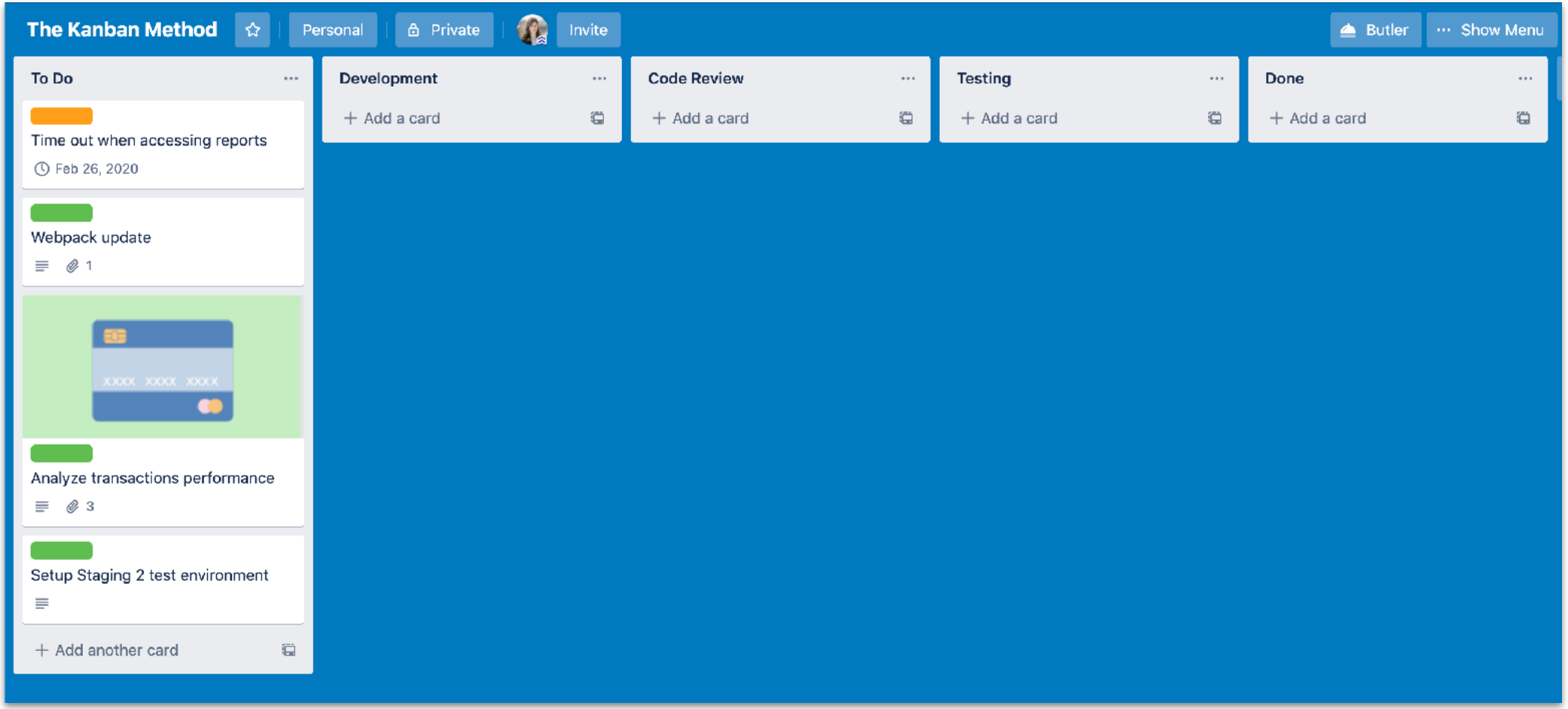


https://www.youtube.com/watch?v=iVaFVa7HYj4&t=116s

Kanban

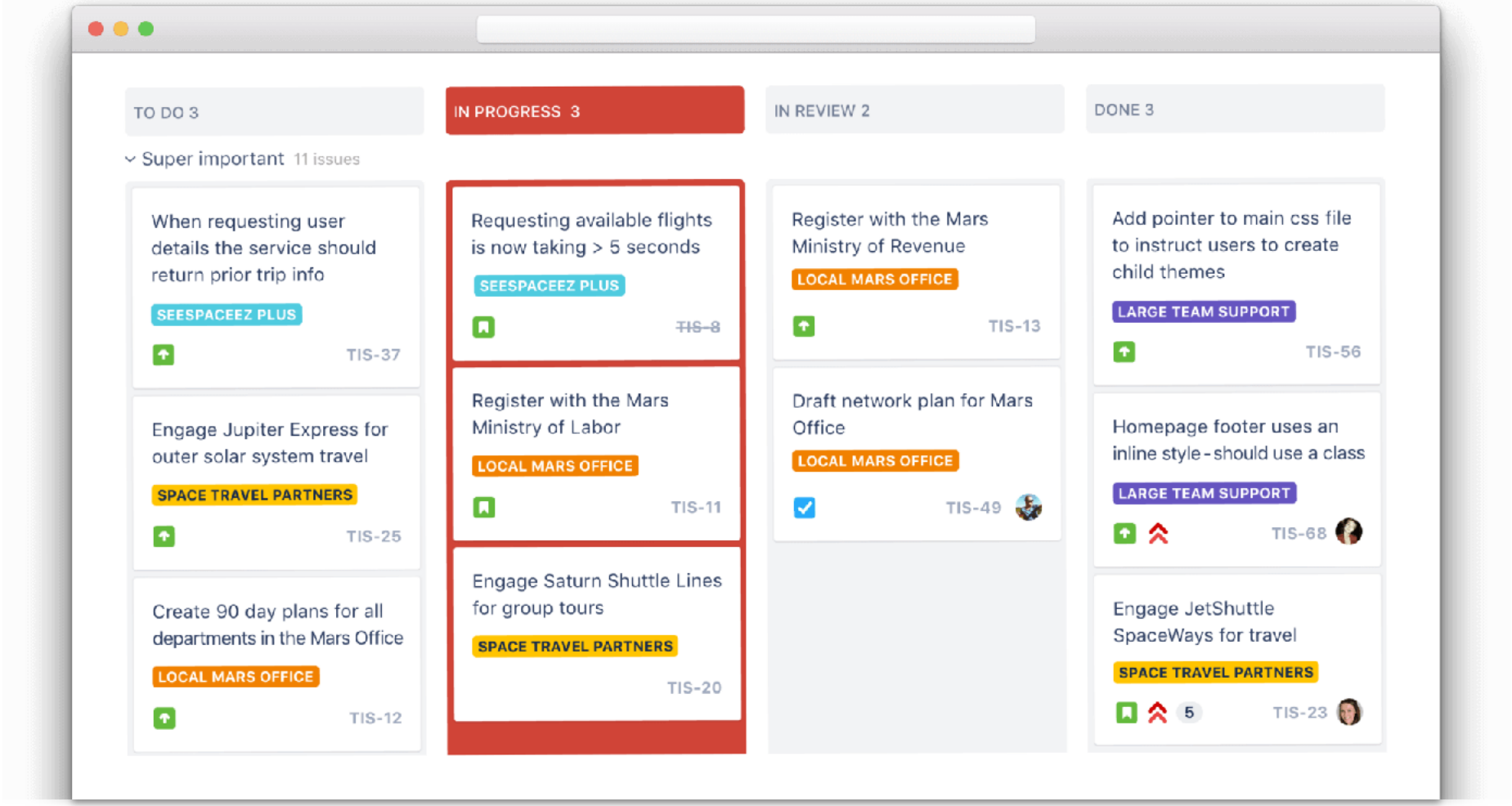
- Work-in-progress (WIP) doesn't deliver value until deployed
- Limiting / minimising WIP makes it easier to identify inefficiencies and constraints in a team's workflow
- Kanban boards clearly visualise WIP and match to team's capacity
- Simple, but effective approach to help teams coordinate

Kanban – Virtual Boards



https://trello.com/en

Kanban – Virtual Boards



https://www.atlassian.com/software/jira

Software Development Processes

Traditional

- Waterfall
- RUP

Agile

- Extreme Programming
- Kanban
- Scrum

Scrum



*"**scrum**"*

- borrowed from rugby (formation of players)

- term introduced to emphasise teamwork

We will follow the **scrum process** for the IS212 project

Scrum

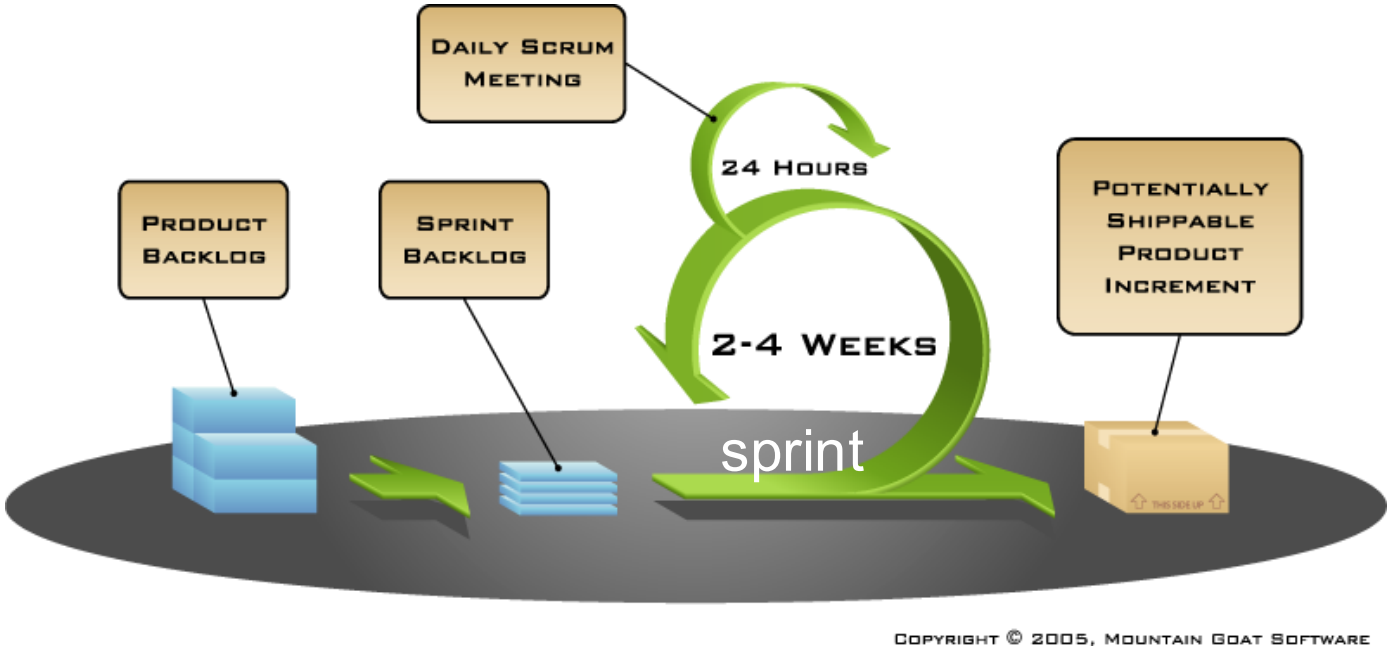


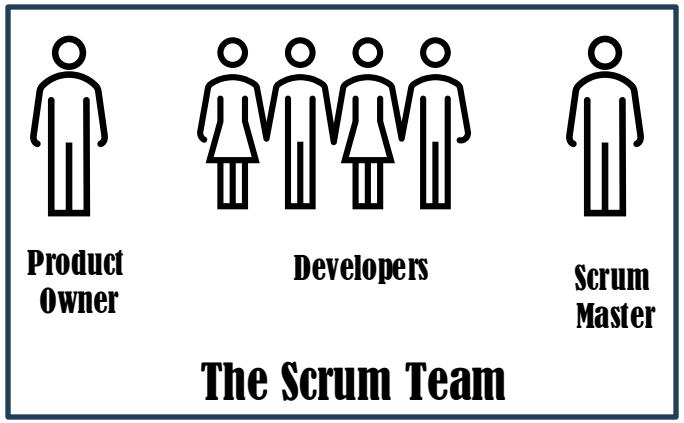
https://www.youtube.com/watch?v=TRcReyRYIMg

Scrum

- Scrum is a lightweight, iterative software process

- The iterations are structured "sprints", which are designed to foster collaboration, accountability, and continuous improvement





Product owner and scrum masters are not project managers. There are only these 3 roles. Only 1 scrum master and 1 product owner. A scrum master can be a developer at the same time but not a product owner. Same for product owner

What is a Product Backlog?

- A product backlog is a **prioritised list** of features that:
  - *Will deliver an overall **Product Goal***
  - *Adds **Value** to the customer*
  - *Is **Estimated** (i.e., amount of effort required)*
- It mainly contains items that bring direct value to customers
  - *Features expressed as User Stories **(see Week 2)***
- It can also contain items that bring indirect value to customers
  - *Enhancements and fixes*
  - *Infrastructure set up (e.g. Git repository, CI/CD pipeline)*
  - *Exploratory research work*
- It does not contain **low-level tasks**
  - *e.g. "send email to Rajesh" ✗, "implement PersonDAO class" ✗*
- It is created / groomed by the Product Owner
- It is a living document that continuously evolves throughout the project





Product Backlog Example **(More in Weeks 2–3)**

Product Goal: To provide a convenient way for shoppers to purchase products online

Priority	User Story	Estimate	More Details (Jira Link)	Status
1	As a shopper, I want to browse the catalog of products, so that I know what to buy	3	https://shopmore.atlassian.net/browse/JIRA-10	**Done (Sprint 1)**
2	As a shopper I want to add a product to my cart, so that I can buy the product	8	https://shopmore.atlassian.net/browse/JIRA-23	**Done (Sprint 1)**
3	As a shopper, I want to remove a product from my cart so that I don't buy the product	5	https://shopmore.atlassian.net/browse/JIRA-15	**In-Prog (Sprint 2)**
4	As a shopper, I want to checkout my cart so that I complete my shopping	13	https://shopmore.atlassian.net/browse/JIRA-09	**In-Prog (Sprint 2)**
7	As a shopper I want to choose payment on delivery, so that can pay after receiving my product	1	https://shopmore.atlassian.net/browse/JIRA-14	**To Do**



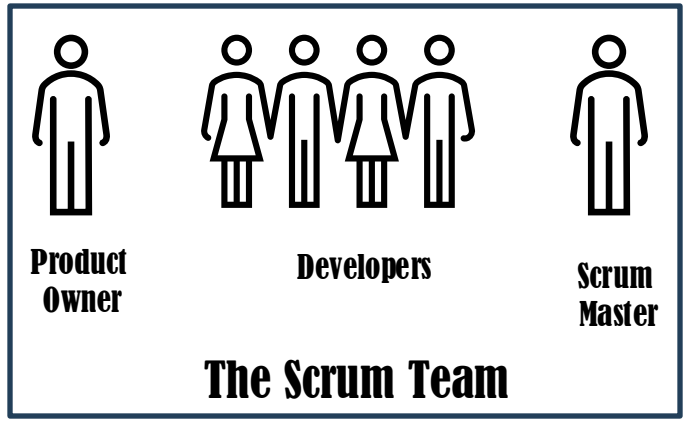
Scrum – Team Members ("Accountabilities")

Developers

- Design, code, test, and integrate the software product
- Deliver high-quality shippable software increments
- Self-organised and cross-functional
- Pace development in a sustainable manner

Scrum Master

- Facilitates scrum meetings (standups, reviews, retros)
- Removes impediments / blockers
- Facilitates mentorship and conflict resolution
- In small teams, may also take on development tasks





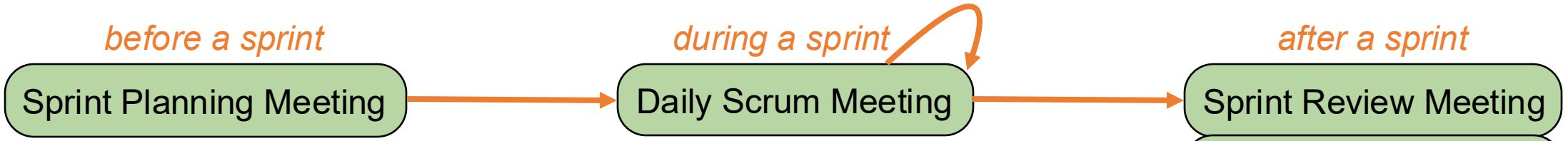
Product Owner

- Identifies and (re-)prioritises product features
- Writes user stories and 'grooms' the product backlog
- Collaborates with stakeholders (e.g. the customer)
- Validates product increments after each sprint

Scrum – Sprints

- Sprints are fixed-length (e.g. 2, 3, or 4 week) iterations that aim to achieve an agreed sprint goal, e.g. a particular software increment
  - For the IS212 project, we recommend choosing a sprint length of **1 or 2 weeks**
  - Sprint goal is frozen for the length of a sprint

- Sprints are lightly managed using "time-boxed" scrum meetings

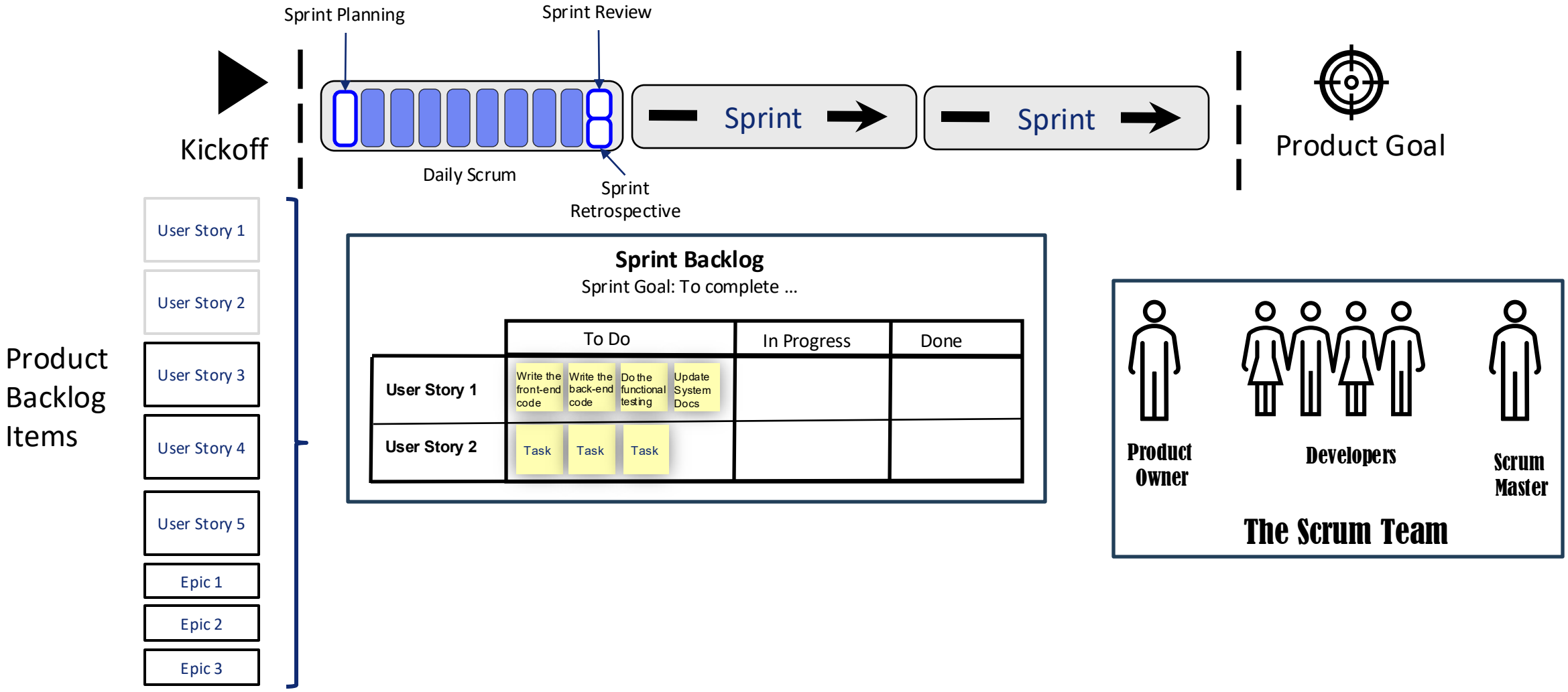


- agree sprint goal based on priorities set by Product Owner
- plan which product backlog items the sprint will tackle (in order to achieve that goal)

- short (~15 mins); facilitated by scrum master
- identify impediments to progress
- **NOT** about finding out who is behind schedule

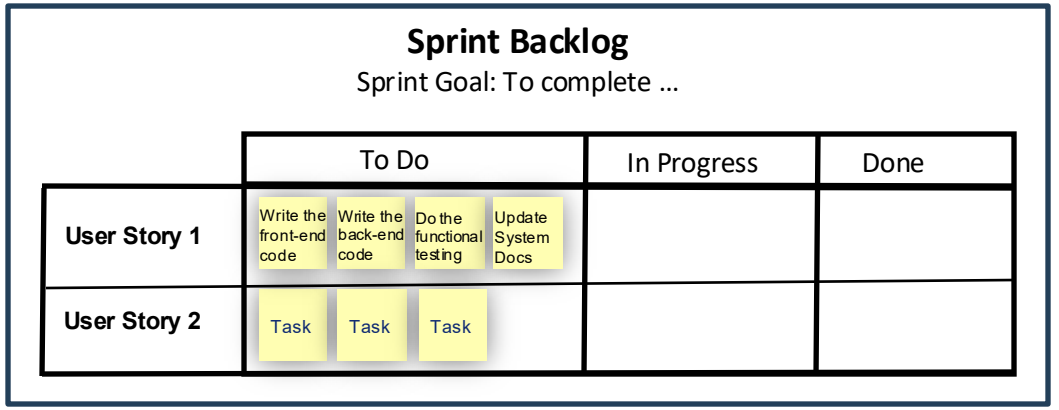
Details of these scrum meetings coming in **Week 3** of IS212

Scrum – Overall Picture **(More in Weeks 2–3)**



Sprint Backlog

- The sprint backlog lists the product backlog items (e.g. user stories) to be completed in the current sprint



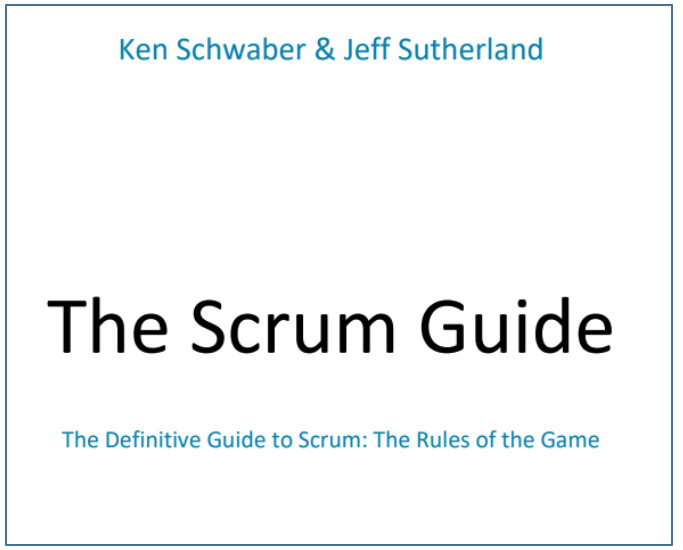
- List of user stories is typically "frozen" for the duration of the sprint. **(Why?)**

- The stories are associated with one or more implementation tasks
  - *E.g. set up database, write front-end code, write tests, write documentation, …*

- The tasks are placed on a scrum taskboard to visualise progress
  - *Updated by team during stand-up meetings*
  - *Monitored by scrum master – stagnant WIP a warning sign!*

How to get Scrum right

- You will use the scrum process for the IS212 project (starting in Week 4)
- You are encouraged to learn by doing – get your hands dirty, make mistakes in sprint #1, reflect, and then improve
- But we also provide a scrum guide and primer as required reading (see eLearn)
- Read them as we 'deep dive' into user stories, estimation, and sprint meetings in **Weeks 2–3**





Scrum – Certification

- The Scrum Alliance provides a number of certification routes for those keen to develop their agile credentials



https://www.scrumalliance.org/

Scrum – Impact

- Scrum has turned iterative development into a precise discipline
  - *Codified goals, duration, and management of iterations (sprints)*
- Savvy marketing operation – certifications turn scrum learners into scrum supporters
- Scrum / sprints have quickly become the industry standard iteration model – even in projects beyond software development
- IS212 teaches one interpretation of the scrum rules. In practice, many real-world teams may 'tweak' them in various ways
  - *Or adapt scrum ideas into other processes, e.g. daily standups in waterfall*

Scrum vs. Kanban – similar boards, but **different**

- Scrum is a different process to Kanban despite the similar boards 

	Scrum	Kanban
Cadence	Fixed-length sprints	Continuous flow
Release methodology	End of each sprint	Continuous delivery
Accountabilities / Roles	Product owner, scrum master, scrum team	No required roles
Key metrics	Velocity	Lead time, cycle time, WIP
Change philosophy	Requirements frozen during sprints	Requirements can change at any point
Visualisation	Scrum board cleared after every sprint	Kanban board used throughout project

Group Exercise

Google Drive Exercise

In groups, copy and complete the template at:

http://smu.sg/XXX

Be ready to share and discuss with us!

Traditional

- Waterfall
- RUP

Agile

- Extreme Programming
- Kanban
- Scrum

Reminder – *Reflections*

- Please start filling in your weekly reflections now
- Complete them by two days after this class so that we have time to read them before we next meet ☺
- **http://smu.sg/is212-reflections-2025**

